

Introduction to Regression

- Regression is a supervised machine learning technique used to predict a continuous numerical value based on one or more input features.
- the goal of regression is to find the relationship between independent variables(features) and a dependent variable (target).

Example:

- Predicting house prices
- Predicting Employee Salaries
- Predicting Stock Prices
- Predicting Sales Revenue

In []:

1.Linear Regression

- Linear Regression is the simplest regression algorithm that models the relationship between input and output variables using a straight line.
- It assumes that a linear relationship exists between the independent variable and dependent variable.

Objective:

- Find the best-fitting straight line that

Example:

- $\text{Salary} = 5000 \times \text{Experience} + 2000$
- if experience = 5years
- $\text{Salary} = (5000 \times 5) + 20000 = 45000$

Why do we Need a Cost Function?

- Initially, the model guesses values for slope(m) and intercept(b).
- these guesses

Mean Squared Error (MSE)

- The Most Common Cost Function for Linear Regression is MSE.

Formula

- $\text{MSE} = \frac{\text{Error}^2 + \text{Error}^2 + \text{Error}^2}{n}$
- $\text{Error} = \text{Actual Value} - \text{Predicted Value}$

Steps

1. Find the Error for Each Prediction.
2. Square each error.
3. Add all Squared errors.
4. Divide by the total number of observations.

Example | Actual | Predicted | Error | Error^2 | |-----|-----|-----|-----| | 10
 | 8 | 2 | 4 | | 15 | 14 | 1 | 1 | | 20 |

Gradient Descent

- Gradient Descent is a method used to find the best line by reducing the prediction error step by step.

Real-Life Example

- Imagine you are on top of a mountain and want to reach the lowest point.
1. Take a Small Step downward.
 2. Check if you're getting lower.
 3. Keep taking small Steps down.
 4. Eventually, you reach the bottom.

In Machine Learning

- Start with random values for slope and intercept.
- Make Predictions.
- Calculate the error using cost function.
- Adjust the values slightly to reduce the error.
- Repeat until the error becomes very small.

```
In [30]: import numpy as np

# Dataset
X=np.array([1,2,3,4])
y=np.array([3,5,7,9])

# Initial Values
m=0 # Start with random guesses
b=0

learning_rate=0.01 # Controls how much the model updates each step.
epochs=1000 # Number of iterations

n=len(X)

for i in range(epochs):

    # Predictions
    y_pred=m*X+b # Current Line Equation.

    # Gradients
    dm= (-2/n)*np.sum(X*(y-y_pred)) # How much to change slope
    db= (-2/n)*np.sum(y-y_pred) # How much to change interce
```

```

    # Update Parameters
    m=m-learning_rate*dm
    b=b-learning_rate*db           # Move toward Lower Error.

    print("Slope (m):",m)
    print("Intercept (b):",b)

```

Slope (m): 2.0048610156782556
Intercept (b): 0.9857080211211781

```

In [31]: import pandas as pd
         from sklearn.linear_model import LinearRegression
         from sklearn.model_selection import train_test_split
         #Sample Data

         df=pd.DataFrame({
             'Experience':[2,1,3,7,4,5,6],
             'Salary':[35000,30000,40000,60000,45000,50000,55000]
         })

         #Input Feature (X)
         X=df[['Experience']]

         # Target Variable (y)
         y=df['Salary']

         # Create Model
         model=LinearRegression()

         # Train Model (OLS happens Here)
         model.fit(X,y)

         # Predict Salary
         predictions=model.predict(X)

         # Results
         print("Slope:",model.coef_[0])
         print("Intercept:",model.intercept_)
         print("Predicted Salaries:",predictions)

         # When fit() runs, OLS Automatically calculates the best slope and

```

Slope: 5000.0
Intercept: 25000.0
Predicted Salaries: [35000. 30000. 40000. 60000. 45000. 50000. 55000.]

```

In [32]: new_experience=pd.DataFrame({'Experience':[8,9,10]})
         new_predictions=model.predict(new_experience)
         print("Predicted Salaries for 8,9,10:")
         print(new_predictions)

```

Predicted Salaries for 8,9,10:
[65000. 70000. 75000.]

```

In [33]: import matplotlib.pyplot as plt
         plt.figure(figsize=(8,5))

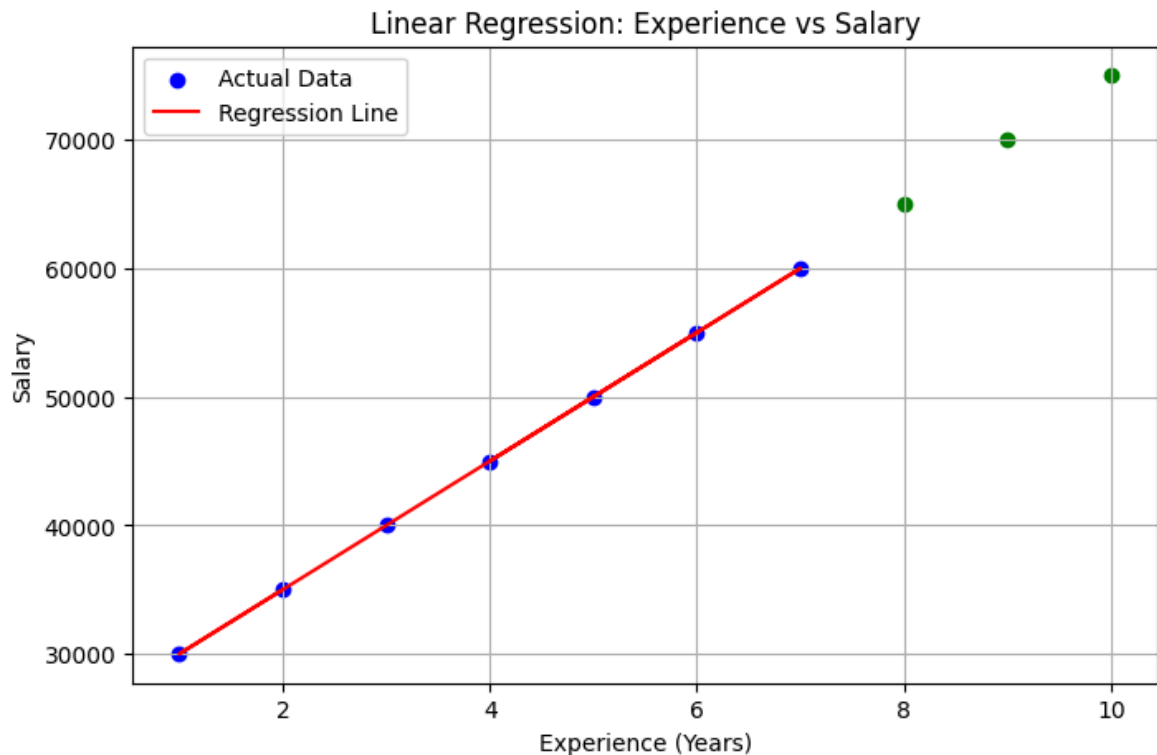
         # Original data points
         plt.scatter(df['Experience'], df['Salary'], color='blue', label='Actual D

```

```

# Regression line (trained data)
plt.plot(df['Experience'], predictions, color='red', label='Regression Li
# New predictions
plt.scatter(new_experience['Experience'], new_predictions, color='green')
# Labels
plt.xlabel("Experience (Years)")
plt.ylabel("Salary")
plt.title("Linear Regression: Experience vs Salary")
plt.legend()
plt.grid()
plt.show()

```



OLS (Ordinary Least Squares)

- when we draw a regression line, there can be many possible lines.
- OLS helps us find the best line.

How?

1. Predict Values using a line.
2. Compare Predicted Values with Actual Values.
3. Find the Error.
4. Square all errors.
5. Add them together.
6. Choose the line with the smallest total error.

Multiple Regression

- Multiple Regression is a Regression technique that uses two or more independent variables to predict a dependent variable.

Example:

- Predicting a house price using:
 1. Area
 2. Number of Bedrooms
 3. House Age
- Instead of one feature, we use multiple features.

```
In [34]: import pandas as pd
from sklearn.linear_model import LinearRegression

# Sample Data
df = pd.read_csv("C:/Users/mahim/Downloads/eda and ml dataset/Salary_data")
print(df.head())
```

	Unnamed: 0	YearsExperience	Salary
0	0	1.2	39344.0
1	1	1.4	46206.0
2	2	1.6	37732.0
3	3	2.1	43526.0
4	4	2.3	39892.0

```
In [35]: # Input Feature (X)
X = df[['YearsExperience']]

# Target Feature (y)
y = df['Salary']

# Create Model
model = LinearRegression()

# Train Model (OLS happens here)
model.fit(X, y)

# Predict Salary
predictions = model.predict(X)

# Results
print("Slope:", model.coef_[0])
print("Intercept:", model.intercept_)
print("Predictions:", predictions)
```

```
Slope: 9449.962321455074
Intercept: 24848.203966523208
Predictions: [ 36188.15875227  38078.15121656  39968.14368085  44693.12484
158
46583.11730587  53198.09093089  54143.08716303  56033.07962732
56033.07962732  60758.06078805  62648.05325234  63593.04948449
63593.04948449  64538.04571663  68318.03064522  72098.0155738
73988.00803809  75878.00050238  81547.97789525  82492.9741274
90052.94398456  92887.932681  100447.90253816  103282.8912346
108007.87239533  110842.86109176  115567.84225249  116512.83848464
123127.81210966  125017.80457395]
```

```
In [36]: # Actual data
plt.scatter(df['YearsExperience'], df['Salary'],
            color='blue', label='Actual Data')
```

```

# Regression line
plt.plot(df['YearsExperience'], predictions,
         color='red', label='Regression Line')

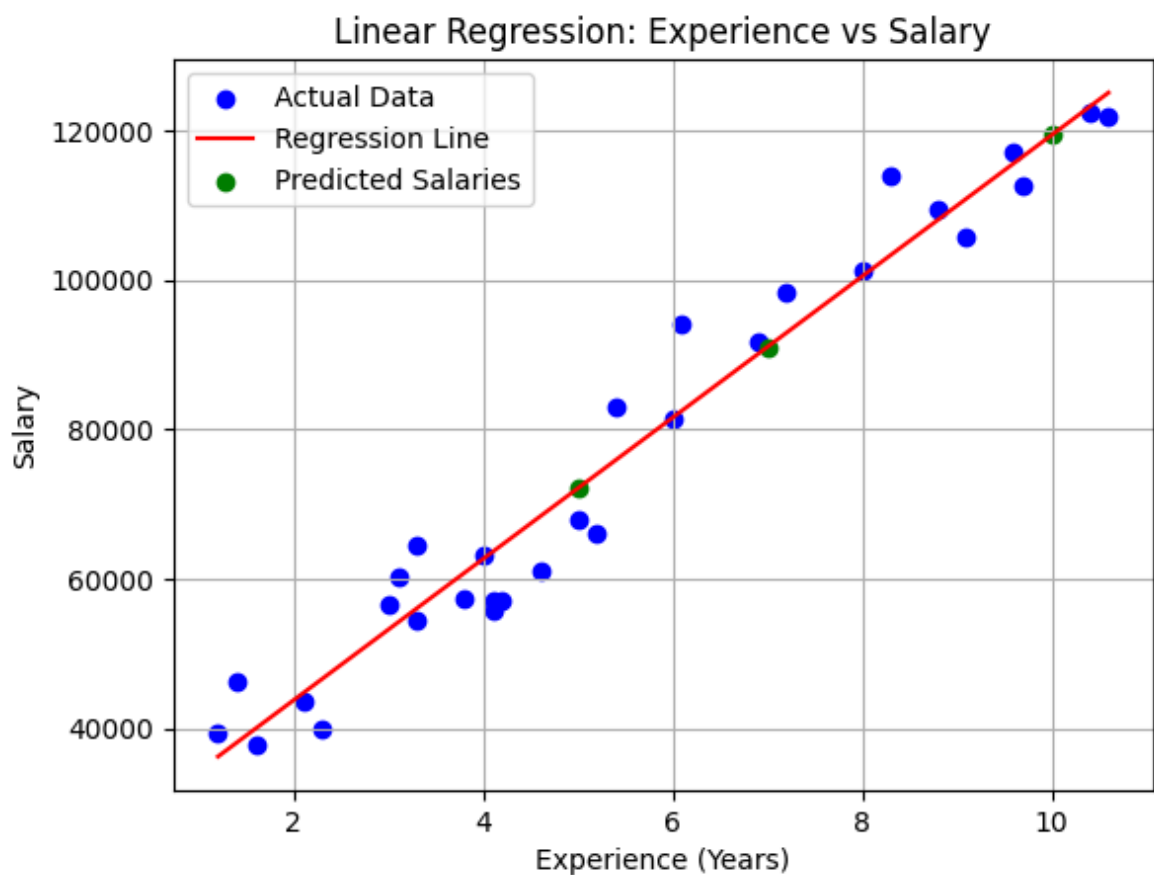
# New experience values
new_experience = pd.DataFrame({
    'YearsExperience': [5, 7, 10]
})

new_predictions = model.predict(new_experience)

# New predicted points
plt.scatter(new_experience['YearsExperience'],
           new_predictions,
           color='green',
           label='Predicted Salaries')

plt.xlabel("Experience (Years)")
plt.ylabel("Salary")
plt.title("Linear Regression: Experience vs Salary")
plt.legend()
plt.grid(True)
plt.show()

```



```

In [37]: df=pd.read_csv("C:/Users/mahim/Downloads/eda and ml dataset/kc_house_data
print(df.head())

```

	id	date	price	bedrooms	bathrooms	sqft_living
0	7129300520	20141013T000000	221900.0	3	1.00	1180
1	6414100192	20141209T000000	538000.0	3	2.25	2570
2	5631500400	20150225T000000	180000.0	2	1.00	770
3	2487200875	20141209T000000	604000.0	4	3.00	1960
4	1954400510	20150218T000000	510000.0	3	2.00	1680

	sqft_lot	floors	waterfront	view	...	grade	sqft_above	sqft_basement
0	5650	1.0	0	0	...	7	1180	
1	7242	2.0	0	0	...	7	2170	4
2	10000	1.0	0	0	...	6	770	
3	5000	1.0	0	0	...	7	1050	9
4	8080	1.0	0	0	...	8	1680	

	yr_built	yr_renovated	zipcode	lat	long	sqft_living15
0	1955	0	98178	47.5112	-122.257	1340
1	1951	1991	98125	47.7210	-122.319	1690
2	1933	0	98028	47.7379	-122.233	2720
3	1965	0	98136	47.5208	-122.393	1360
4	1987	0	98074	47.6168	-122.045	1800

	sqft_lot15
0	5650
1	7639
2	8062
3	5000
4	7503

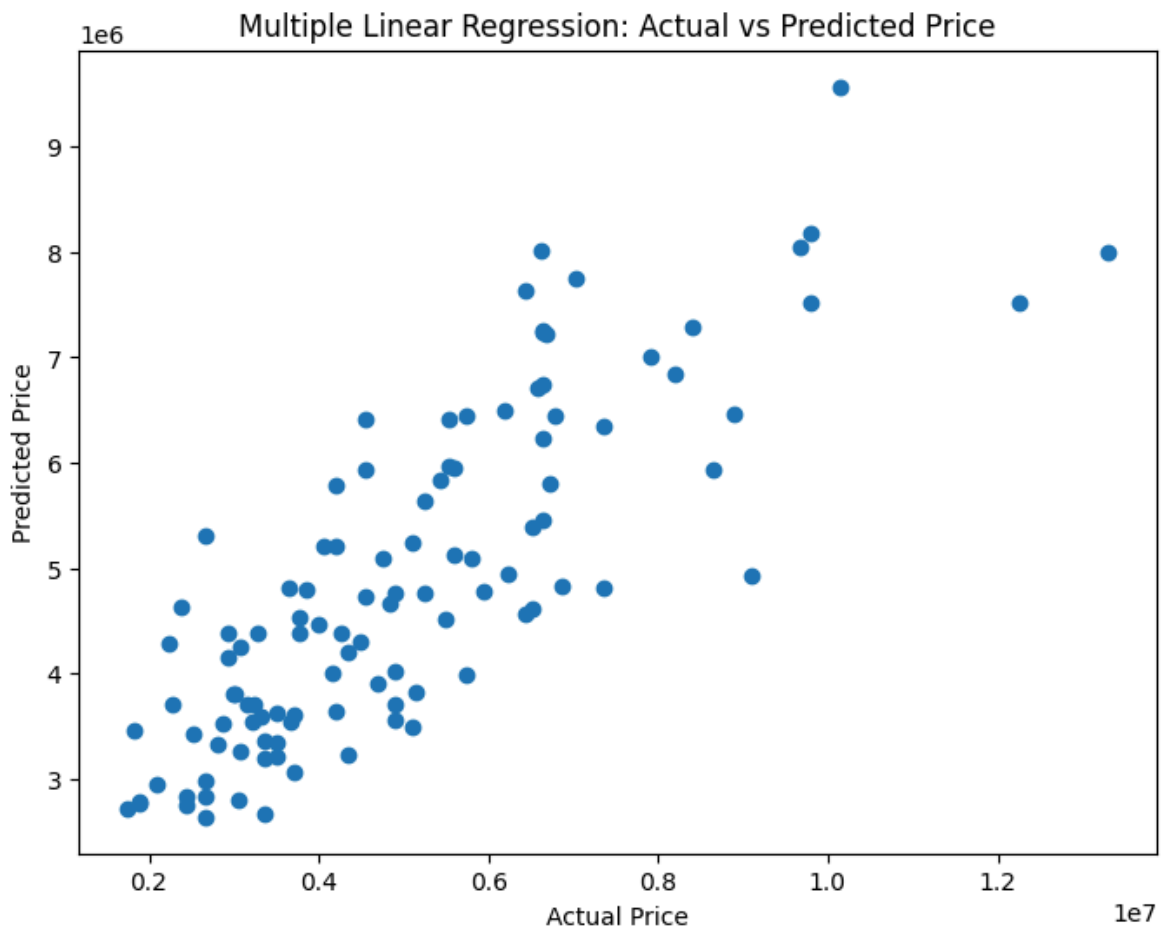
[5 rows x 21 columns]

```
In [44]: X = df.drop('price', axis=1)
y = df['price']
# Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
# Create and train model
model = LinearRegression()
model.fit(X_train, y_train)
# Predictions
y_pred = model.predict(X_test)

# Evaluation
print("Intercept:", model.intercept_)
print("Coefficients:", model.coef_)
plt.figure(figsize=(8, 6))
plt.scatter(y_test, y_pred)
plt.xlabel("Actual Price")
plt.ylabel("Predicted Price")
plt.title("Multiple Linear Regression: Actual vs Predicted Price")
plt.show()
```

Intercept: -127711.16739244293

Coefficients: [2.35848772e+02 7.85744924e+04 1.09711727e+06 4.06223164e+05
3.66824192e+05 2.33146766e+05 3.93159779e+05 6.87881311e+05
7.85550579e+05 2.25756514e+05 6.29901661e+05 2.10397118e+05]



3. Polynomial Regression

- Polynomial Regression is an extension of Linear Regression that captures non-linear relationships by adding polynomial terms.

why polynomial regression?

- Not all relationships are straight lines.

Examples:

1. Age vs Income
2. Advertising Spend vs Sales
3. Temperature vs Electricity Consumption

- These Relationships

-

```
In [1]: import pandas as pd  
  
import numpy as np  
  
import matplotlib.pyplot as plt
```



```
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression

# Sample dataset
df = pd.DataFrame({
    'Hours': [1, 2, 3, 4, 5],
    'Marks': [10, 25, 45, 70, 100]
})

X = df[['Hours']]
y = df['Marks']

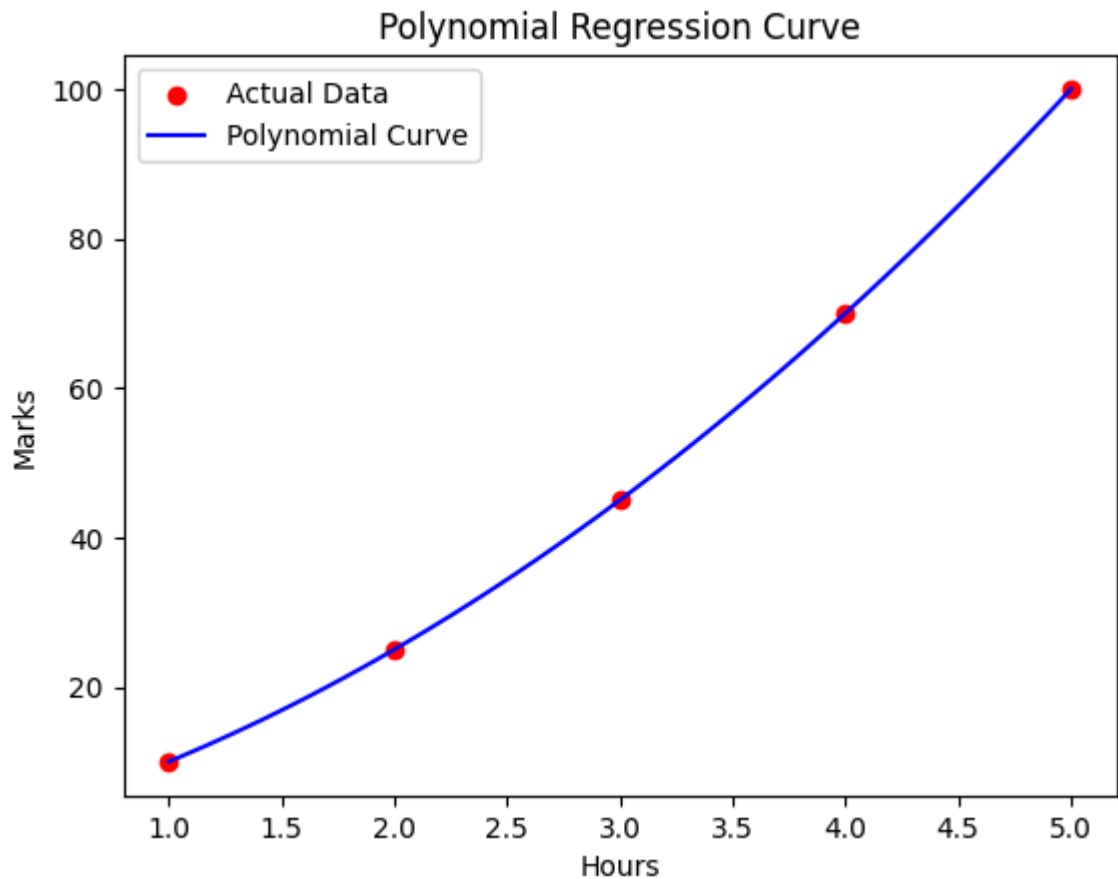
# Polynomial transformation
poly = PolynomialFeatures(degree=4)
X_poly = poly.fit_transform(X)

# Model training
model = LinearRegression()
model.fit(X_poly, y)

# Smooth curve generation (IMPORTANT STEP)
X_range = np.linspace(X.min(), X.max(), 100).reshape(-1, 1)
X_range = pd.DataFrame(X_range, columns=['Hours'])
X_range_poly = poly.transform(X_range)

# Predictions for curve
y_curve = model.predict(X_range_poly)

# Plotting
plt.scatter(X, y, color='red', label='Actual Data')
plt.plot(X_range, y_curve, color='blue', label='Polynomial Curve')
plt.title('Polynomial Regression Curve')
plt.xlabel('Hours')
plt.ylabel('Marks')
plt.legend()
plt.show()
```



```
In [2]: import numpy as np

import matplotlib.pyplot as plt

from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression

X = np.array([1,2,3,4,5,6,7,8,9,10]).reshape(-1,1)
y = np.array([2,5,8,15,28,35,50,60,85,95])

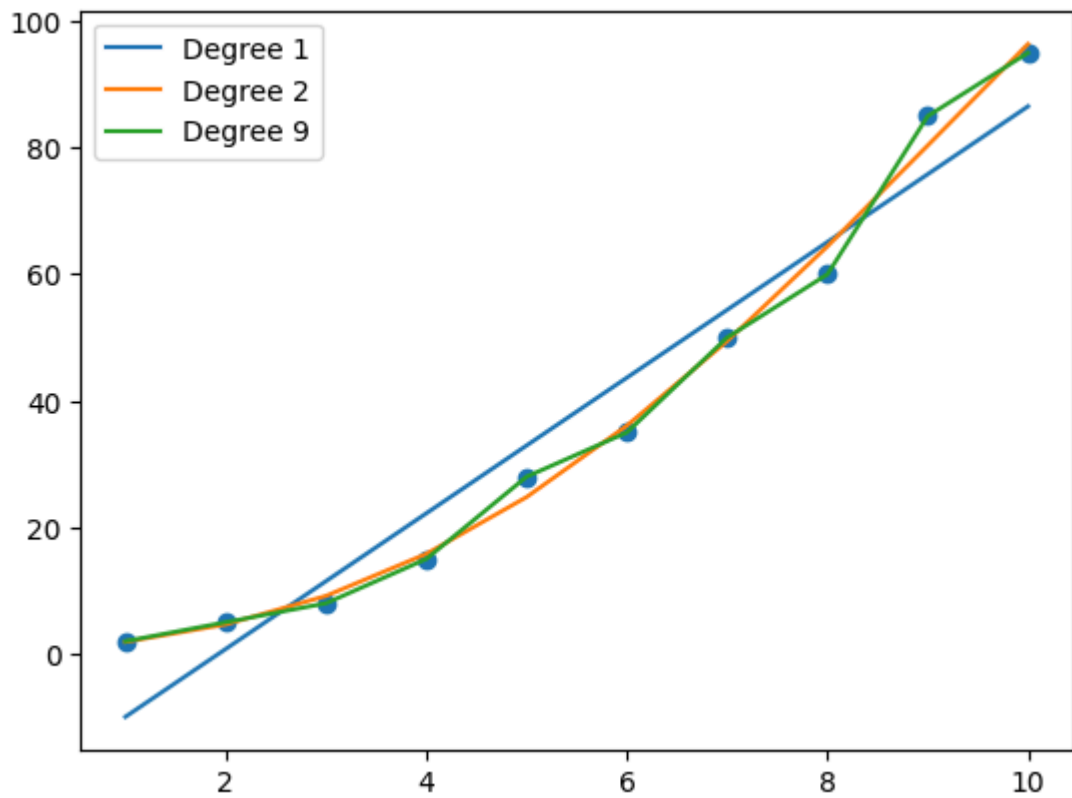
# Degree 1
poly1 = PolynomialFeatures(degree=1)
X1 = poly1.fit_transform(X)

# Degree 2
poly4 = PolynomialFeatures(degree=4)
X4 = poly4.fit_transform(X)

# Degree 9
poly9 = PolynomialFeatures(degree=9)
X9 = poly9.fit_transform(X)

model1 = LinearRegression().fit(X1,y)
```

```
model4 = LinearRegression().fit(X4,y)
model9 = LinearRegression().fit(X9,y)
plt.scatter(X,y)
plt.plot(X, model1.predict(X1), label="Degree 1")
plt.plot(X, model4.predict(X4), label="Degree 2")
plt.plot(X, model9.predict(X9), label="Degree 9")
plt.legend()
plt.show()
```



In [1]: *##Ridge Code*

```
from sklearn.linear_model import Ridge
from sklearn.datasets import make_regression
X, y = make_regression(

    n_samples=100,

    n_features=5,

    noise=20,

    random_state=42

)
ridge = Ridge(alpha=1.0)
ridge.fit(X, y)
print("Coefficients:")
print(ridge.coef_)
```

Coefficients:
[62.4844981 98.07649112 57.01070157 52.31272567 35.35343002]

In [2]: *##Lasso Code*

```
from sklearn.linear_model import Lasso
from sklearn.datasets import make_regression
X, y = make_regression(
    n_samples=100,
    n_features=10,
    noise=20,
    random_state=42
)
lasso = Lasso(alpha=2.0)
lasso.fit(X, y)
print("Coefficients:")
print(lasso.coef_)
```

Coefficients:
[17.88753444 54.76076704 0. 60.9632585 90.0800089 68.30461063
 81.35787563 4.03519677 2.86610797 67.99906953]

In [3]: *##ElasticNet Code*

```
from sklearn.linear_model import ElasticNet
from sklearn.datasets import make_regression
X, y = make_regression(
    n_samples=100,
    n_features=10,
    noise=20,
    random_state=42
)

elastic = ElasticNet(
    alpha=3.0,
    l1_ratio=0.5
)
elastic.fit(X, y)
print("Coefficients:")
print(elastic.coef_)
```

Coefficients:

[10.65990986 22.10890939 0.49979146 25.9235125 33.08321402 34.26097045
35.7929364 -7.32756763 4.65692898 25.41452644]